

Project Verification & Submission Summary

Final Project Submission Summary

****Project 1B: Automated Reconciliation Engine for Multi-Bank Settlement****

1. Metadata

* ****Project Code Name****: ReconOps - Settlement Guardian (v1.0.0)

* ****Submission Date****: 2026-07-02

*

****GitHub**

Repository**:

[AUTOMATED-RECONCILIATION-ENGINE](https://github.com/nikitanehrag24-a11y/AUTOMATED-RECONCILIATION-ENGINE)

* ****Submission Branch****: `master`

* ****Submission Tag****: `v1.0`

* ****Commit Hash****: `203f441799b8e353c0c575a0567bc8fbfcc41e1f`

2. Executive Summary

ReconOps is an enterprise-grade automated transaction reconciliation system. It ingests heterogeneous bank statements, transforms transactions into a standard format, runs a three-tier matching pipeline, flags data anomalies, classifies exceptions into 18 categories, tracks SLA countdowns, logs events in a cryptographic audit trail, and generates statutory reports.

3. Core Technical Features

3.1 Streaming & State-Machine Parsers

* ****CSV Parser****: Uses Python's native streaming `csv` module to handle large files efficiently. Maps custom column names dynamically.

* ****SWIFT MT940 Parser****: Parses bank statement lines (using tags like `:61:` and `:86:`) with a state machine. Resolves formatting quirks like decimal comma separators and missing funds codes.

* ****ISO 20022 CAMT.053 XML Parser****: Processes standard XML namespaces recursively using `lxml.etree` XPath queries.

3.2 Canonical Normalisation Chain

* ****Timezone Alignment****: Converts local transaction times to UTC using the `pytz` library.

* ****Precision Rounding****: standardises financial amounts to 4 decimal places utilizing Banker's Rounding (`ROUND\_HALF\_EVEN`) to eliminate mathematical bias.

* ****Reference & Name Sanitisation****: Strips special characters, expands acronyms, and normalises text cases.

3.3 Three-Tier Matching Engine

1. ****Exact Match****: Hash-based lookup using transaction keys `(txn\_id + amount + currency + direction)` in $O(N+M)$ complexity.

2. ****Fuzzy Match****: Candidate blocking limits comparisons to a ± 2 day window and identical amounts. Computes string similarities using `RapidFuzz` Jaro-Winkler and Token Set Ratio.

3. ****Rule-Based Match****: Handles clearing offsets, split transactions, and netted settlements utilizing a depth-bounded subset-sum backtracking algorithm.

3.4 Exception Classification & Escalations

* ****18 Exception Categories****: Maps unmatched items to distinct categories (e.g. `DATE\_MISMATCH`, `FEE\_DEDUCTION`, `REVERSAL\_PENDING`).

- * **30-Minute Reversal SLA**: Flags direction reversals (credit/debit swapped) with a critical **30-minute SLA** and immediate escalation to Tier 4 Compliance.
- * **4-Tier Escalation Model**: Escalates open exceptions across queues upon SLA breaches or value thresholds exceeding INR10 Lakh.

3.5 Cryptographic Audit Trail

- * **Tamper-Evident Ledger**: Chains audit logs using SHA-256 hashes.
- * **Log Verification**: Re-computes the hash chain to confirm audit trail integrity and detect unauthorized edits.

3.6 Anomaly Detection

- * **Amount Deviation**: Checks if a transaction amount is more than 3 standard deviations from its historical average.
- * **Velocity Alert**: Flags when hourly transaction counts for a counterparty spike over 200% of their daily rolling average.
- * **Absence Alert**: Detects missing daily bank statement files.

4. Technology Stack

- * **Backend Framework**: FastAPI (REST Gateway)
- * **Operations Portal**: Streamlit
- * **Database ORM**: SQLAlchemy (SQLite for development/tests, PostgreSQL for production)
- * **Validation**: Pydantic v2
- * **Deployment**: Docker & Docker Compose

5. Verification & Test Results

Our codebase has a comprehensive test suite (27 unit and integration tests) achieving high test coverage (>80% cumulative coverage). All tests pass successfully:

```
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.6.0
plugins: cov-7.1.0, anyio-4.14.1
collected 27 items

tests/test_exceptions_audit.py::test_exception_classification_format_error PASSED
tests/test_exceptions_audit.py::test_exception_classification_direction_reversal PASSED
tests/test_exceptions_audit.py::test_escalation_manager_sla_breach PASSED
tests/test_exceptions_audit.py::test_cryptographic_audit_log PASSED
tests/test_matching.py::test_string_similarity PASSED
tests/test_matching.py::test_token_set_ratio PASSED
tests/test_matching.py::test_confidence_score_calculation PASSED
tests/test_matching.py::test_exact_matching PASSED
tests/test_matching.py::test_fuzzy_matching PASSED
tests/test_matching.py::test_rule_based_matching_date_offset PASSED
tests/test_matching.py::test_rule_based_matching_split PASSED
tests/test_normalisation.py::test_timezone_normalise PASSED
tests/test_normalisation.py::test_timezone_normalise_date_only PASSED
tests/test_normalisation.py::test_currency_normalise PASSED
tests/test_normalisation.py::test_reference_clean PASSED
tests/test_normalisation.py::test_amount_standardise_bankers_rounding PASSED
tests/test_normalisation.py::test_counterparty_clean PASSED
tests/test_normalisation.py::test_normalise_record_end_to_end PASSED
tests/test_parsers.py::test_csv_parser_valid PASSED
tests/test_parsers.py::test_csv_parser_split_reference_axis PASSED
tests/test_parsers.py::test_mt940_parser_valid PASSED
```

```
tests/test_parsers.py::test_mt940_parser_deviations PASSED
tests/test_parsers.py::test_camt053_parser_valid PASSED
tests/test_api.py::test_upload_file_api PASSED
tests/test_api.py::test_upload_duplicate_file_prevention PASSED
tests/test_api.py::test_trigger_reconciliation_api PASSED
tests/test_api.py::test_dashboard_and_audit_api PASSED

===== 27 passed, 40 warnings in 7.88s =====

---
```

6. How to Run & Verify

Running via Docker Compose

To start the services in the background:

```
docker compose up --build -d
```

Access points:

- * **FastAPI REST API Docs**: ``http://localhost:8000/docs``
- * **Streamlit Operations Portal**: ``http://localhost:8501``

Running the Test Suite

```
python -m pytest -v
```

7. Proprietary License Notice

****STRICTLY PRIVATE & CONFIDENTIAL****

This software, related documentation, data specifications, and concepts are the exclusive property of ****Zetheta Algorithms Private Limited****. Copying, public redistribution, sharing on public code platforms, or social media publication is strictly prohibited.